

Kotlin Coding Challenge – Address Book

1) Overview

Build a small **console application in Kotlin** that models a simple address book. The goal is to assess your ability to write **idiomatic Kotlin**, structure a small codebase cleanly, and work with a relational database (either **plain JDBC** or an **ORM**).

What we're looking for

- Solid Kotlin fundamentals (data classes, null-safety, collections, extension functions, sealed types/enum classes, etc.)
 - Clean, modular design with clear boundaries
 - Pragmatic testing
 - Sensible SQL schema design and JDBC usage
-

2) Functional Requirements

Implement a CLI that allows users to **create, read, update, delete (CRUD)** contacts.

Each **Contact** consists of:

- **Personal:** `firstName`, `lastName`, `birthDate`
- **Addresses** (one or more): `street`, `houseNumber`, `postalCode`, `city`, `type` (e.g., `BUSINESS`, `PRIVATE`)
- **PhoneNumbers** (one or more): `number`, `type` (e.g., `LANDLINE`, `MOBILE`, with `BUSINESS/PRIVATE`)

Required CLI actions:

- Add a contact (with at least one address or one phone number)
 - **Required fields:**
 - **Contact:** `firstName`, `lastName`
 - **Address:** `street`, `houseNumber` (other fields optional)
 - **PhoneNumber:** all fields required (`number` + `type`)
- Edit a contact
- Delete a contact (and related data)
- List all contacts (compact overview)
- Show a single contact (detailed view)
- **Basic search** by last name (exact or prefix match)

UX note: Keep the CLI ergonomic and robust against invalid input.

3) Persistence

- Use a **relational database**. We suggest **PostgreSQL** for the challenge.
- You may choose one of two approaches:
 1. **Plain JDBC** (no ORM), or
 2. An **ORM** such as **Hibernate/JPA** (or **Exposed** for Kotlin).
- Regardless of approach:
 - Create a **sensible schema** with appropriate keys and relationships.
 - Keep a clear boundary so the **data layer is swappable** (e.g., a `Repository/Port` interface with either a JDBC or ORM adapter).
 - Document your choice (JDBC vs. ORM) and the trade-offs in the README.

Note: If you prefer a lightweight local setup, SQLite/H2 is fine for development, but please target PostgreSQL semantics where relevant (types, constraints).

4) Kotlin-Specific Engineering Expectations

Keep it **idiomatic and pragmatic**. We don't expect you to showcase every Kotlin feature.

- Use **data classes** for your domain models.
- Apply **null-safety** thoughtfully; validate inputs.
- Use **enums or sealed types** where they make the model clearer.
- Favor **clear, simple code** over clever tricks.

Optional: Show layering (domain ↔ persistence ↔ CLI) if it helps structure the app.

5) Testing

- Use **JUnit 5** or **Kotest**. For mocking, **MockK** is fine (optional).
- Write **unit tests** for the most important domain logic (e.g., validation, mapping, repositories with an in-memory or temp DB).

6) Non-Functional Requirements

- Provide a **README** with:
 - How to build/run/tests (Gradle commands)
 - DB choice & schema overview
 - Key design decisions and trade-offs
- Use Gradle (Kotlin DSL preferred).
- Apply a formatter/linter (e.g., **ktlint** or **ktfmt**). CI is a plus but not required.

Target time box: ~6–8 hours of focused work. Don't over-engineer; pick reasonable compromises and document them.

7) Bonus (optional, only if you have time)

- Export a single contact as **JSON** (pretty printed)
- **Sorting & filtering** in the list view (e.g., by last name, by city)
- **Migrations** with Flyway or Liquibase
- Import from JSON (basic happy path)

Bonuses are nice but **not required**. Don't sacrifice core quality for extras.

8) Deliverables

- Public GitHub repository (do not set to private) including:
 - Source code
 - Tests
 - README with instructions
 - SQL/migration scripts if applicable

We will clone the repo, run `./gradlew build` and launch the app from the CLI following your README.

9) Evaluation Rubric

We score along these axes:

1. **Correctness & Completeness** (does it meet the core requirements?)
 2. **Kotlin Proficiency** (idiomatic use of language features)
 3. **Design & Architecture** (clear layering, testability, separation of concerns)
 4. **Data Modeling** (reasonable schema, integrity, constraints)
 5. **Testing Quality** (useful tests with clear intent)
 6. **Developer Experience** (README, setup friction, code clarity)
-

10) Suggested Tech Stack

- **Kotlin** (JDK 17+)
- **Gradle** (Kotlin DSL)
- **PostgreSQL** (suggested); SQLite/H2 acceptable for local dev
- **JDBC driver** and either:
 - **Plain JDBC**, or

- **ORM: Hibernate/JPA (or Exposed)**
- **JUnit 5** or **Kotest**, optional **MockK**
- Optional: **ktlint/ktfmt**